Journal of King Saud University – Engineering Sciences 32 (2020) 330–338

Contents lists available at ScienceDirect

Journal of King Saud University – Engineering Sciences

journal homepage: www.sciencedirect.com

Original article

Tabu search and particle swarm optimization algorithms for two identical parallel machines scheduling problem with a single server

Ibrahim Alharkan [a], Mustafa Saleh [a,b,*], Mageed A. Ghaleb [a,b], Husam Kaid [a], Abdulsalam Farhan [a], A. Almarfadi [a]

[a] Industrial Engineering Department, College of Engineering, King Saud University, P. O. Box 800, Riyadh 11421, Saudi Arabia
[b] Industrial & Manufacturing Systems Engineering, Taiz University, P.O. Box 6803, Taiz, Yemen

ARTICLE INFO

Article history:
Received 19 September 2018
Accepted 21 March 2019
Available online 22 March 2019

Keywords:
Scheduling
Identical parallel machines
Single server
Tabu search
PSO

ABSTRACT

This paper proposes two efficient algorithms, which are tabu search and particle swarm optimization, for scheduling two identical parallel machines with a single server. The server has to set up the relevant machine before starting job processing. The objective is to minimize the makespan. This problem is considered unary NP-hard problem. The performance of the two proposed algorithms is evaluated using previously solved instances from the literature. The instances were solved using three different algorithms, which are genetic algorithm (GA), simulated annealing algorithm (SA) and I-L algorithm. We used the results of these three algorithms as a benchmark to compare with the two new introduced algorithms, which are tabu search (TS) and geometric particle swarm optimization (GPSO). The obtained results show that the proposed algorithms have a great performance for large instances. Moreover, the obtained results are very close to a lower bound, and in some instances, an optimal solution is achieved. In addition, TS performs better than SA and GA in term of average makespan for large instances and outperforms all algorithms in term of reaching the lower bound for all instances greater than 200 jobs, while GPSO comes second.

1. Introduction

A need to share common server is a widespread problem in manufacturing. A server can be, for example, a robot, a worker, or a tool by which machine setup/load are carried out before starting job processing. Sharing server resource leads to machine idle time. Therefore, it is important to handle such issue in scheduling problems. This article considered the problem of scheduling identical parallel machines with a common single server. In this problem, there are n independent jobs, a single server, and two identical parallel machines M_i, where $i = 1, 2$. Each job j has processing time p_j and set-up time s_j (or equivalently loading time). A server sets up or loads each job j onto a machine M_i before job j being processed. Each job is processed immediately after the setting up/loading. All set-ups (loadings) have to be performed by a single server, which can handle at most one job at a time on only one machine. The problem is denoted as $P2, S1||C_{max}$ using standard scheduling notation. This model has a wide range potential application areas in the manufacturing environment such as automated material handling systems, semiconductor industry, or scheduling of maintenance and setups systems with a limited number of staff (Kim and Lee, 2012), textile industry (Allahverdi, Ng, Cheng, and Kovalyov, 2008), or network computing (Glass, Shafransky, and Strusevich, 2000), and so on.

The problem $P2, S1||C_{max}$ was first introduced by Kravchenko and Werner (1997). The problem with equal setup times ($P2, S1|s_j = s|C_{max}$) is considered strongly NP hard problem by Hall, Potts, and Sriskandarajah (2000). Moreover, Abdekhodaee, Wirth, and Gan (2006) showed that the general problem of makespan minimization ($P2, S1||C_{max}$) is NP-hard in the strong sense. Due to the practicality of the considered problem, several algorithms and formulations were developed to solve the problem in its general form as well as several special cases. For example, Abdekhodaee and

* Corresponding author at: Industrial Engineering Department, College of Engineering, King Saud University, P. O. Box 800, Riyadh 11421, Saudi Arabia.
E-mail addresses: aimalhark@ksu.edu.sa (I. Alharkan), emustafas87@gmail.com (M. Saleh), mghaleb@ksu.edu.sa (M.A. Ghaleb), yemenhussam@yahoo.comd (H. Kaid), afarhan@ksu.edu.sae (A. Farhan).

Peer review under responsibility of King Saud University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.jksues.2019.03.006
1018-3639/

Wirth (2002) proposed an integer programming formulation to solve small-size problems up to 12 jobs using CPLEX. It has been reported that computation time was excessive for large-scale problems. Therefore, two special cases were considered, which are short processing times and equal length jobs. Moreover, two simple backward/forward $O(n \log n)$ heuristics were also proposed to find near optimal solutions for the problem in its general form $(P2, S1||C_{max})$. Gan, Wirth, and Abdekhodaee (2012) proposed two mixed integer linear programming (MILP) formulations and two variants of a branch-and-price scheme for instances of $n = 8, 20, 50,$ and 100. Hasani, Kravchenko, and Werner (2014a) proposed MILP formulations based on decomposing a schedule into a set of blocks to solve the $P2, S1||C_{max}$ problem. The proposed formulations were applied to instances with up to 250 jobs. Kim and Lee (2012) presented MILP formulations and a hybrid heuristic combining simulated annealing and tabu search for instances up to 40 jobs (the runtime was within 3600 s). Abdekhodaee et al. (2006) developed GA, greedy heuristic, and a version of the Gilmore–Gomory algorithm to the general case of the problem with instances of 50 and 100 jobs. Hasani, Kravchenko, and Werner (2014b) proposed SA and GA algorithms to solve the problem with large-scale instances up to 1000 jobs within a time limit of 14,400 s. Hasani, Kravchenko, and Werner (2016) developed an iterated local search algorithm named I-L algorithm to solve the problem with large-scale instances up to 1000 jobs. The performance of the developed algorithm (I-L algorithm) was compared to SA and GA. Arnaout (2017) Proposed an ant colony optimization (ACO) algorithm for the problem and assessed its performance with branch and bound (up to 100 jobs). ACO algorithm results were also compared with GA and SA algorithms from Hasani, Kravchenko, and Werner (2014b) with instances up to 1000 jobs. It was claimed that ACO outperforms other algorithms in large problems.

The average gap from a designated lower bound, computational complexity, and long running time are the issues countered in the previous studies. Therefore, there is a need to develop more efficient and effective algorithms to the problem under investigation. The objective of this paper is to propose two efficient metaheuristics to solve the single-server scheduling problem of n independent jobs on two identical parallel machines to minimize makespan considering arbitrary processing and setup times. The proposed metaheuristics are tabu search (TS) algorithm and particle swarm optimization (PSO) algorithm. The performance of the two proposed algorithms was compared to the performance of benchmark algorithms from the literature, which are SA, GA and I-L algorithms.

Including this introductory section, the paper is organized as follows. Section 2 provides a problem statement and some basic concepts. The TS and PSO algorithms are presented in Section 3. In Section 4, instances preparation, parameter tuning, computational results, and comparisons are introduced. Finally, Section 5 presents the conclusion and future work.

2. Problem description

The problem considered in this paper is to schedule a given set of n jobs on two identical parallel machines with a single server. The following assumptions have been considered:

- A set of jobs n has to be processed by two identical parallel machines M_1 and M_2.
- Each job j has to be loaded and then processed at once of the two identical machines.
- Both the server and the machine are required for set-up time units s_j and loading process, respectively.
- The set-up time s_j and processing time $p_j$for each job j are known in advance.
- Each job needs a single process on one of the two machines.
- Each machine and server can perform only one job at a time, and all processes are non-preemptive.
- Server, machines, and all jobs should be available at time zero,

The objective is to minimize the makespan (C_{max}). The considered problem is NP-hard problem and can be expressed using Graham's notations as follows$P2, S1|s_j|C_{max}$. The following notations will be used to define the considered problem:

- n: Number of jobs.
- m: Number of machines.
- j: Job's index $(j = 1, \cdots, n)$.
- i: Machine's index $(i = 1, 2)$.
- p_j: Processing time of job j.
- s_j: Setup time of job j.
- $st_j(setup)$: Starting time for setup for job j.
- C_j: Completion time of job j, $C_j = st_j(setup) + s_j + p_j$.
- C_{max}: Maximum completion time (Makespan).

The proposed objective function for considered scheduling problem under investigation is stated as follows:

$$Minimize\ C_{max} = \max_{j \in n} C_j \tag{1}$$

The schedule is constructed based on the machine and server availability. The following example illustrates the schedule construction of the problem. Consider a set of eight jobs with the setup and processing times as shown in Table 1. Let the current job sequence S_0 be$\{3, 1, 8, 5, 2, 4, 6, 7\}$. According to the given sequence, the jobs are assigned in a greedy manner to the nearest available machine. Before operating a job on the machine, the server should prepare it.

From the set S_0, job 3 is selected to be operated first and assigned to machine M_1 or M_2 with the same chance since both machines are available on the starting point $(t = 0)$, here job 3 is assigned to M_1. The server has to prepare machine M_1, which takes five time units (S_3), before loading job 3. Once the server complete preparing machine M_1, job 3 is processed immediately for six time units $(P3)$. Meanwhile, since machine M_2 is available, the server prepares it to receive the second selected job in S_0 that is job 1; the preparation of machine M_2 takes four time units (S_1). The server will complete setting-up machine M_2 for job 1 at point $t = 9$ on the timeline and then job 1 will be operated on machine M_2 for 6 time units. The server will wait until one of the two machines is available to receive the third job in S_0 (job 8). At point $t = 11$ on the timeline, machine M_1 is available. The same procedure will continue until all jobs in S_0 are scheduled.

In this way, the constructed schedule is jobs $\{3, 8, 2, 6\}$ on machine M_1 and jobs $\{1, 5, 4, 7\}$ on machine M_2. The constructed schedule is presented on a Gantt chart as shown in Fig. 1. The parts with grey shades are the set-ups and the makespan value (C_{max}) is 35.

From the previous example, we can notice that two types of waiting could occur in the schedule, which could be that the server is waiting for one of the machines to be available or that one of the machines waiting for the server to be available. These two issues (the two types of waiting) can be found in Fig. 1. For instance, in periods (9–11 and 13–15), the server waits for machines M_1 and M_2, respectively, to be available. The same in period (16–17), machine M_1 waits for the server to be available.

In order to evaluate the quality of the generated schedules (solutions), a lower bound that is developed by Gan et al. (2012) was considered. The considered lower bound can be explained as follows:

Table 1
Process and set-up times of an example.

j	1	2	3	4	5	6	7	8
s_j	4	3	5	3	2	1	3	2
p_j	6	5	6	4	3	4	5	3

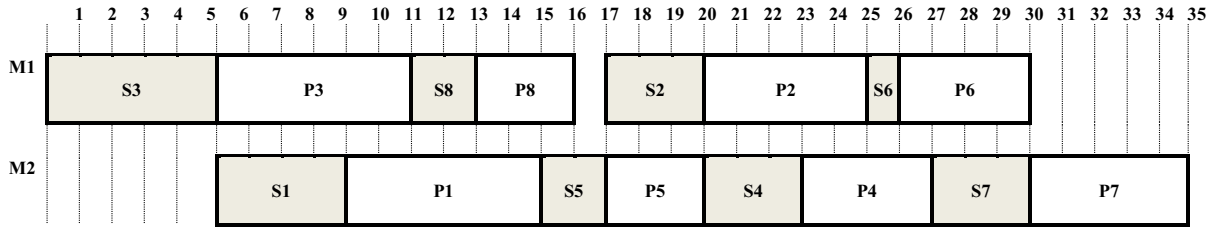

Fig. 1. Gantt chart of an example.

$$LB = \max\{LB_1, LB_2, LB_3\} \tag{2}$$

where:

$$LB_1 = \frac{1}{2}\left(\sum_{i\in J}(s_i + p_i) + min_{i\in J}\{s_i\}\right) \tag{3}$$

$$LB_2 = \sum_{i\in J}(s_i) + min_{i\in J}\{p_i\} \tag{4}$$

$$LB_3 = min_{i\in J}\{s_i + p_i\} \tag{5}$$

3. Proposed algorithms

The proposed solution methods are based on TS and PSO approaches. They are categorized based on the algorithms applied for job sequencing and machine assignment. With reference to the algorithms presented in this paper, the job sequences are generated using the stochastic search process of TS and PSO. The jobs are then allocated to machines as described in the previous example (see Section 2).

3.1. Tabu search

The tabu search algorithm was introduced by Glover (1989) and since then it becomes very popular in solving optimization problems. TS uses memory to store information related to the search process, which helps in avoiding cycles (previous visited solutions could be selected again). The used memory is called tabu list. At each iteration of the TS, the tabu list is updated. The updating process is simply conducted by deleting the last element of the list and inserting the new element at the top of it. The risk of rejecting solutions that have not yet been generated may be occur because of the restrictions in the tabu list. Therefore, tabu solutions are checked for some condition, which is called aspiration criteria. If the tabu solution has an objective value that is better than the best objective value found so far, then this tabu solution will be accepted and its move will be removed from the tabu list (Talbi, 2009). As any local search method, TS has several elements that needed to be tuned before starting the search process. The main elements of the TS are initial solution, neighboring structure, tabu list, aspiration criteria, and stopping criteria. The initial solution to TS was set using the longest processing time (LPT) rule, in which jobs are sequenced in non-increasing order of the processing time. Pairwise interchange and backward-shifted reinsertion proposed by Della Croce, Narayan, and Tadei (1996) are two classical and useful operators to construct promising neighborhood structures. For generating new candidate solutions, we applied the pairwise interchange operator in the proposed TS algorithm. A pair of jobs were randomly selected and then the positions of these jobs were interchanged (i.e., swapped) to generate new candidates. The number of generated candidates was set to 70 candidates based on preliminary experiments. Moreover, based on the same preliminary experiments, the size of the tabu list was set to seven elements and the first-in-first-out (FIFO) strategy was applied to update tabu list. The aspiration criteria was set as previously mentioned (if a move results in a solution that is better than the best solution found so far). Finally, the stopping criteria was set so the computational times (in seconds) in all algorithms are the same, or when reaching the lower bound. In order to compare the performance of the TS to the other algorithms proposed by Hasani et al. (2016), the stopping condition was set to the same values as in Hasani et al. (2014a), in which it was set to $(\frac{300}{8}) * n$ seconds for the small-scale instances and 3600 s for the large scale instances. Assuming a minimization problem, the general TS algorithm can be outlined as follows:

Algorithm 1: TS algorithm template.

$s = s_0$; %Initial solution
Initialize the tabu list, medium-term and long-term memories;
Repeat
  Generate a set "A" of solutions;
  Find best neighbor s' of A; %non tabu or aspiration criterion holds
  $s = s'$;
  Update tabu list and aspiration conditions;
  If $f(s) < f(s_0)$ **Then** $s_0 = s$;
Until Stopping criteria satisfied
Output: s_0 is the best-found solution.

In general, Reinelt and Gerhard (1994) stated that the basic TS difficulties include the tabu list design, the mechanism of list management, and the non-prohibited move selection. This is not a straightforward task for some optimization problems. Moreover, TS may be very sensitive to some parameters such as the size of the tabu list (Talbi, 2009). A very good discussion of TS applications on scheduling and many other problems can be found in the book of Glover and Laguna (2013).

3.2. Particle swarm optimization

The PSO algorithm was proposed by Kennedy and Eberhart (1995), motivated by such social behaviors of organisms, as the flocking together of birds, the schooling together of fish, or even the behaviors of human (Kennedy and Eberhart, 1995; Siddhartha et al., 2012). PSO was first introduced to optimize various continuous nonlinear functions. Recently, PSO algorithms have been successfully used in a wide range of applications such as traveling salesman problem (Wang and Huang, 2003), vehicle routing (Zhao et al., 2004), scheduling (Jia, Qu, and Wang, 2016). However, the continuous nature of the PSO algorithm is the major obstacle of its successful application to our problem. To remedy this drawback, several researchers suggested different ways to encode a schedule (or generally a permutation) to PSO particle, which lead to different PSO variates to solve the problem. An important discrete version of the PSO algorithm that is first introduced by Moraglio, Di Chio, Togelius, and Poli (2008) for solving constrained combinatorial problems is the geometric PSO (GPSO). GPSO connects between PSO and evolutionary algorithms. A new crossover operator was introduced that simply generalize PSO to virtually any combinatorial solution representation.

In the standard PSO, each particle i is defined by its velocity v_i and position x_i at time t (i.e., iteration$t$). The velocity $v_i(t)$ explains the change of the particle's position $x_i(t)$ from an iteration to the next. The particle will adjust its position using information about its new velocity $v_i(t)$ and previous position $x_i(t-1)$. In order to obtain the new velocity $v_i(t)$, the following information is needed: the previous velocity $v_i(t-1)$ of the particle, the particle's best position (this is named as the particle local best $x_L(t-1)$, which is the best position this particle achieved so far) and the overall best position defined by all the particles (this is named as the global best position x_G, which is the best position achieved by all particles so far). This leads to a two-step updating proses in the standard form of PSO, which are velocity update and position update.

The key issue in the GPSO is the direct update of the particle's position instead of using the two-step update, also the velocity concept is avoided. Using a new developed crossover operator, named three-parent mask-passed crossover operator (3PMX), the GPSO can easily update the position of the particle using the following equation:

$$x_i(t) = 3PMX((x_i(t-1), w_1), (x_L(t-1), w_2), (x_G, w_3)) \quad (6)$$

where,

- The factor t represents the iterative generation number.
- The particle $x_i(t)$ corresponds to a permutation $\pi_i(t)$.
- x_L is the local best position of a particle.
- x_G is the global best position among all particles.
- $3PMX$ is the three-parent mask-passed crossover operator that developed by Moraglio et al. (2008).
- $w_1, w_2, and w_3$ are weight values indicate for each element in the mask, the probability of having values from the parents x_i, x_L, or x_G, respectively.

As noticed, three parents (as inspired from evolutionary algorithms) are incorporated in the update process of any particle. The following numerical example explains the position update process proposed in the GPSO. Consider the new position of a certain particle is to be obtained at a certain iteration t. The previous position $x_i(t-1)$ is represented by the following sequence {3, 1, 2, 4, 5}. The local best position $x_L(t-1)$ is {5, 1, 3, 2, 4} and the global best position x_G is {1, 5, 4, 2, 3}. The weight values are 0.2, 0.4 and 0.4 for$w_1, w_2, and w_3$, respectively. The new position $x_i(t)$ will be generated as follows:

Step 1: generate the mask vector, which is the same length as the sequence and its elements are randomly generated with values of 1, 2 or 3 (because we have three parents). The generated mask and the three parents are listed below.

Mask	3 1 2 2 3
$x_i(t-1)$	3 1 2 4 5
$x_L(t-1)$	5 1 3 2 4
x_G	1 5 4 2 3
$x_i(t)$	*xxxxx*

Step 2: starting by the mask first element from the right (3), the first element in parent 1 (x_i) and parent 2 (x_L) should be equal to the first element of parent 3 (x_G). To do that the first and the second elements in the first two parents will be swapped. The result will be as follows:

Mask	3 1 2 2 3
$x_i(t-1)$	1 3 2 4 5
$x_L(t-1)$	1 5 3 2 4
x_G	1 5 4 2 3
$x_i(t)$	1*xxxx*

Similarly, based on the second value of the mask's element, the second element of parents two (x_L) and three (x_G) should be equal to the second element of parent one (x_i). Again, elements two and three in parent two will be swapped, and in parent three, elements two and five will be swapped. The results will be as follows:

Mask	3 1 2 2 3
$x_i(t-1)$	1 3 2 4 5
$x_L(t-1)$	1 3 5 2 4
x_G	1 3 4 2 5
$x_i(t)$	13*xxx*

The same process will be continued until we reach the last mask's element. The new position generated will be $x_i(t) = \{1, 3, 5, 2, 4\}$.

Then, the particle's new position is updated using a mutation operator to produce a new particle at the neighborhood of the resulted particle's position from Eq. (6). Based on the concept of GPSO, the proposed PSO algorithm is stated as follows:

Algorithm 2: GPSO algorithm template.

Random initialization of the whole swarm (as permutations);
Repeat
  Evaluate $f(x_i)$;
  For all particles i;
    Update and move to the new position using the 3PMX:
    $x_i(t) = 3PMX((x_i(t-1), w_1), (x_L(t-1), w_2), (x_G(t-1), w_3))$;
    Mutate x_i;
    If $f(x_i) < x_L$, ***Then*** $x_L = x_i$;
    If $f(x_i) < x_G$, ***Then*** $x_G = x_i$;
    Update (x_i);
End For**Until** Stopping criteria**Output:** Best solution found.

As explained earlier, GPSO has a set of input parameters. The main input parameters are the swarm size and the weights $(w_1, w_2, and w_3)$. The other inputs are the mutation operator and

Table 2
Parameters for GPSO.

Parameters	Considered Values	Selected Value
Swarm size	30, 50 and 70	70
Mutation type	Swap and Insertion	Swap
w_1	0.1, 0.20 and 0.3	0.2
w_2	0.3, 0.4 and 0.5	0.4
w_3	0.3, 0.4 and 0.5	0.4

stopping condition. The swarm size and the weights are taken from Moraglio et al. (2008). We fine-tuned the parameters around the considered values by Moraglio et al. (2008), because we are solving a different problem. Three values are considered, which are the same value and a combination of smaller and larger values. A full factorial design was used to determine the best value for each parameter form sets of considered values. The considered values and the selected values (based on the conducted computational experiments) are presented in Table 2. The used mutation operator was the swap operator (two positions on the sequence are selected at random, and values are swapped), while the stopping condition was set as explained in TS algorithm in Section 3.1.

For the sake of brevity, the details of the GPSO, the 3PMX operator and the basic PSO are not given here, but the reader can refer to Moraglio et al. (2008) for a detailed example of how the 3PMX operator works and more information about GPSO, as well as Talbi (2009), for examples and more information about basic PSO.

4. Results and discussion

The evaluating of the performance of the proposed TS and GPSO algorithms is conducted by computational experiments. The proposed algorithms have been coded using MATLAB software and the computational experiments for all instances have been conducted on the same computer with following specifications. Processor: Intel (R) Core™ i7-4702MQ CPU at 2.2 GHz; RAM: 16 GB.

In order to evaluate the performance of the proposed algorithms (TS and GPSO), an experimental study is conducted using a set of benchmark instances. The same instances were solved in different previous studies (Gan et al., 2012; Hasani et al., 2014a, b, 2016). As stated in Hasani et al. (2016), the instances were generated randomly based on two factors, which are number of jobs (n) and the server's load value (L). Several values were considered for these two factors. For the number of jobs the considered values were $n \in \{8, 20, 50, 100, 200, 250, 300, 400, 500, 600, 700, 800, 900,$ and 1000$\}$. On the other hand, the considered values for the server's load value were $L \in \{0.1, 0.5, 0.8, 1, 1.5, 1.8, 2\}$. Processing times ($p_j$) were uniformly distributed in the interval [0,100], and setup times (s_j) were distributed uniformly in the interval $[0, 100 \times L]$. For each value of L, five instances were generated for each value of n, except when $n \in \{8, 20\}$, 10 instances were generated.

The performance of the two proposed algorithms is compared to three benchmark algorithms that are proposed by Hasani et al. (2016) for the same instances. The three-benchmark algorithms are based on simulated annealing (SA), genetic algorithms (GA) and an algorithm named I-L. based on the proposed lower bound, an error can be used to assess the performance of the proposed algorithms. The error can be defined as follows:

$$Error = C_{max}/LB \tag{7}$$

The computational results of the two proposed algorithms (TS and GPSO) along with the three benchmark algorithms (SA, GA and I-L) from Hasani et al. (2016) are given in Table 3. Results are also presented in Figs. 2 and 3. The following notations are used in Table 3:

Table 3
Computational results with GA, SA, I-L, TS, and PSO for all instances size.

n	Algorithm	L	0.1	0.5	0.8	1	1.5	1.8	2
8	SA	R_{avg}	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
		R_{max}	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
	GA	R_{avg}	1.000000	1.000333	1.000774	1.002032	1.001572	1.001004	1.000000
		R_{max}	1.000000	1.003333	1.004938	1.020316	1.01087	1.01004	1.000000
	I-L	R_{avg}	1.063851	1.08376	1.103816	1.079312	1.034216	1.032276	1.019124
		R_{max}	1.162436	1.144475	1.219753	1.173033	1.097826	1.180722	1.101361
	TS	R_{avg}	1.003653	1.011138	1.030459	1.029766	1.011566	1.012695	1.001644
		R_{max}	1.013953	1.041667	1.054598	1.063758	1.075175	1.064103	1.011662
	GPSO	R_{avg}	1.004764	1.011475	1.031847	1.035704	1.012052	1.013764	1.001644
		R_{max}	1.013953	1.041667	1.060345	1.067391	1.075175	1.074786	1.011662
20	SA	R_{avg}	1.000000	1.000398	1.008599	1.013818	1.006818	1.002574	1.003781
		R_{max}	1.000000	1.002786	1.037223	1.032646	1.048281	1.011898	1.015952
	GA	R_{avg}	1.000000	1.000398	1.008252	1.013614	1.006818	1.002551	1.003781
		R_{max}	1.000000	1.002786	1.037223	1.033505	1.048281	1.011898	1.015952
	I-L	R_{avg}	1.022241	1.035102	1.067376	1.080847	1.032863	1.020372	1.016807
		R_{max}	1.049073	1.060932	1.125754	1.141383	1.138258	1.045398	1.044609
	TS	R_{avg}	1.000000	1.000259	1.008701	1.013977	1.007378	1.002627	1.003829
		R_{max}	1.000000	1.001393	1.038229	1.032646	1.051939	1.011898	1.015952
	GPSO	R_{avg}	1.000000	1.0012953	1.0119987	1.0173587	1.007448	1.0038231	1.0047487
		R_{max}	1.000000	1.005571	1.0402414	1.0395189	1.049012	1.0152975	1.0208178
50	SA	R_{avg}	1.000000	1.000000	1.004535	1.007519	1.001322	1.000041	1.000000
		R_{max}	1.000000	1.000000	1.021277	1.022306	1.006608	1.000204	1.000000
	GA	R_{avg}	1.000000	1.000000	1.005414	1.008589	1.001322	1.000041	1.000000
		R_{max}	1.000000	1.000000	1.02474	1.025709	1.006608	1.000204	1.000000
	I-L	R_{avg}	1.012915	1.019622	1.039196	1.037914	1.012539	1.004642	1.002518
		R_{max}	1.022812	1.032048	1.081148	1.074102	1.046256	1.014442	1.012020
	TS	R_{avg}	1.000000	1.000000	1.01219	1.012152	1.001322	1.000041	1.000000
		R_{max}	1.000000	1.000000	1.034636	1.028355	1.006608	1.000204	1.000000
	GPSO	R_{avg}	1.000000	1.0008457	1.0108341	1.0123784	1.0017	1.0001798	1.0003191
		R_{max}	1.000000	1.0022087	1.0306779	1.0287335	1.006608	1.0006949	1.0011333

Table 3 (*continued*)

n	Algorithm	*L*	0.1	0.5	0.8	1	1.5	1.8	2
100	SA	R_{avg}	1.000000	1.000000	1.000676	1.007576	1.000679	1.000239	1.000666
		R_{max}	1.000000	1.000000	1.003143	1.019471	1.00257	1.000767	1.002567
	GA	R_{avg}	1.000000	1.000000	1.000766	1.007949	1.000651	1.000239	1.000709
		R_{max}	1.000000	1.000000	1.003592	1.020465	1.00257	1.000767	1.002875
	I-L	R_{avg}	1.008932	1.006095	1.019904	1.032133	1.006771	1.007497	1.004851
		R_{max}	1.014008	1.011242	1.034018	1.051858	1.02172	1.023131	1.015941
	TS	R_{avg}	1.000000	1.000000	1.009473	1.0214	1.000707	1.000239	1.000688
		R_{max}	1.000000	1.000000	1.021554	1.030399	1.00257	1.000767	1.002772
	GPSO	R_{avg}	1.000000	1.0003776	1.005818	1.0151506	1.001461	1.0005957	1.0009364
		R_{max}	1.000000	1.0008226	1.0131357	1.0276177	1.00347	1.0020447	1.0036961
200	SA	R_{avg}	1.000000	1.000026	1.000022	1.002301	1.000391	1.000113	1.00001
		R_{max}	1.000000	1.000131	1.000108	1.004995	1.001485	1.000235	1.000048
	GA	R_{avg}	1.000000	1.000078	1.000086	1.004239	1.000324	1.000124	1.000000
		R_{max}	1.000000	1.000261	1.00021	1.007247	1.001485	1.000235	1.000000
	I-L	R_{avg}	1.002223	1.003501	1.002449	1.022286	1.003083	1.002772	1.000332
		R_{max}	1.004395	1.004975	1.003771	1.029082	1.008698	1.004708	1.001662
	TS	R_{avg}	1.000000	1.000522	1.005627	1.022024	1.000381	1.000113	1.000000
		R_{max}	1.000000	1.000658	1.00701	1.027421	1.001697	1.000235	1.000000
	GPSO	R_{avg}	1.0000343	1.0003147	1.0018357	1.0121999	1.001028	1.0003384	1.0001407
		R_{max}	1.0001714	1.00059	1.0034419	1.0152136	1.004102	1.0007311	1.0005539
250	SA	R_{avg}	1.000000	1.000000	1.000228	1.008543	1.000098	1.000036	1.000068
		R_{max}	1.000000	1.000000	1.000345	1.017036	1.000267	1.000136	1.0003
	GA	R_{avg}	1.000000	1.000064	1.000681	1.011189	1.000097	1.000036	1.00006
		R_{max}	1.000000	1.000109	1.000915	1.017679	1.000267	1.000136	1.0003
	I-L	R_{avg}	1.001274	1.003112	1.012326	1.019115	1.001843	1.002747	1.000195
		R_{max}	1.003335	1.006223	1.035282	1.023918	1.003279	1.005391	1.000973
	TS	R_{avg}	1.000000	1.000803	1.010091	1.0257	1.000064	1.000027	1.000051
		R_{max}	1.000000	1.001094	1.012721	1.034876	1.000267	1.000136	1.000257
	GPSO	R_{avg}	1.0000293	1.0002124	1.0018225	1.0143832	1.000402	1.0001169	1.0000677
		R_{max}	1.0001464	1.0004375	1.00302	1.0204918	1.001013	1.0003636	1.0002999
300	SA	R_{avg}	1.000000	1.000036	1.001526	1.005532	1.000155	1.000117	1.000013
		R_{max}	1.000000	1.000092	1.004777	1.011988	1.000688	1.000219	1.000034
	GA	R_{avg}	1.000000	1.000092	1.004777	1.011988	1.000688	1.000219	1.000034
		R_{max}	1.00012	1.00027	1.008114	1.016639	1.000559	1.000253	1.000068
	I-L	R_{avg}	1.002818	1.001335	1.008061	1.019074	1.001949	1.001801	1.00106
		R_{max}	1.004806	1.002209	1.019244	1.032481	1.005118	1.002465	1.002357
	TS	R_{avg}	1.000000	1.000909	1.011074	1.022913	1.000209	1.000102	1.000007
		R_{max}	1.000000	1.00106	1.015015	1.026924	1.000774	1.000217	1.000034
	GPSO	R_{avg}	1.000000	1.0001423	1.0026153	1.0105259	1.000536	1.0002923	1.0001021
		R_{max}	1.000000	1.0003502	1.0037158	1.013018	1.001144	1.0007952	1.0004441
400	SA	R_{avg}	1.000018	1.000689	1.00304	1.009597	1.000086	1.000000	1.00001
		R_{max}	1.000088	1.001131	1.003608	1.014479	1.000363	1.000000	1.000051
	GA	R_{avg}	1.000018	1.000566	1.004739	1.014526	1.000099	1.000006	1.000015
		R_{max}	1.000088	1.000708	1.006511	1.019455	1.000396	1.000028	1.000051
	I-L	R_{avg}	1.001723	1.002621	1.00548	1.018065	1.001689	1.000017	1.000326
		R_{max}	1.004118	1.003473	1.009137	1.025457	1.006669	1.000085	1.001139
	TS	R_{avg}	1.000000	1.001124	1.010328	1.02474	1.000092	1.000000	1.000000
		R_{max}	1.000000	1.001498	1.011305	1.032192	1.000429	1.000000	1.000000
	GPSO	R_{avg}	1.0000357	1.0002108	1.0012392	1.0121362	1.000197	1.0000221	1.0000301
		R_{max}	1.0000895	1.0002618	1.0016909	1.0194547	1.000792	1.0000544	1.0001011
500	SA	R_{avg}	1.000043	1.002537	1.006854	1.020747	1.000339	1.000082	1.000000
		R_{max}	1.000146	1.004683	1.012574	1.02494	1.001348	1.000319	1.000000
	GA	R_{avg}	1.000057	1.001213	1.01067	1.028368	1.000881	1.000177	1.000035
		R_{max}	1.000073	1.001544	1.016856	1.030537	1.002036	1.000387	1.000157
	I-L	R_{avg}	1.001608	1.001492	1.006849	1.019009	1.000472	1.000312	1.000231
		R_{max}	1.003123	1.002546	1.011298	1.025995	1.001485	1.000987	1.001153
	TS	R_{avg}	1.000000	1.001194	1.010611	1.029505	1.000093	1.000018	1.000000
		R_{max}	1.000000	1.001373	1.016674	1.033677	1.00044	1.000091	1.000000
	GPSO	R_{avg}	1.000000	1.0002734	1.0026852	1.0174286	1.000328	1.0001924	1.0000039
		R_{max}	1.000000	1.0004243	1.005467	1.0195582	1.0011	1.0005465	1.0000196
600	SA	R_{avg}	1.000024	1.002125	1.008182	1.019092	1.000223	1.000055	1.000032
		R_{max}	1.000059	1.002444	1.014993	1.023348	1.000642	1.000202	1.000078
	GA	R_{avg}	1.000023	1.001821	1.01271	1.031281	1.000751	1.000195	1.000082
		R_{max}	1.000059	1.002242	1.017596	1.035234	1.001461	1.000477	1.000167
	I-L	R_{avg}	1.001714	1.001373	1.004262	1.010358	1.001051	1.000683	1.000758
		R_{max}	1.002384	1.001833	1.008633	1.013592	1.003919	1.003322	1.001622
	TS	R_{avg}	1.000000	1.001451	1.010787	1.027274	1.000101	1.000026	1.000013
		R_{max}	1.000000	1.001569	1.012666	1.033121	1.000399	1.000128	1.000033
	GPSO	R_{avg}	1.000000	1.0002017	1.0020553	1.0144409	1.000185	1.0000367	1.0000118
		R_{max}	1.000000	1.0002646	1.0026394	1.0177988	1.000399	1.0001285	1.001622

(*continued on next page*)

Table 3 (*continued*)

n	Algorithm	*L*	0.1	0.5	0.8	1	1.5	1.8	2
700	SA	R_{avg}	1.000122	1.003321	1.011206	1.0258	1.000334	1.000075	1.000014
		R_{max}	1.000155	1.00467	1.012843	1.031786	1.000643	1.000157	1.000028
	GA	R_{avg}	1.00001	1.003183	1.021096	1.045315	1.003747	1.000928	1.000154
		R_{max}	1.000051	1.00379	1.023328	1.052606	1.00615	1.002028	1.000285
	I-L	R_{avg}	1.000748	1.000685	1.002555	1.010412	1.000996	1.000000	1.000233
		R_{max}	1.001239	1.001107	1.006718	1.026745	1.001552	1.000000	1.000726
	TS	R_{avg}	1.000000	1.001535	1.010634	1.027971	1.000094	1.000003	1.000003
		R_{max}	1.000000	1.001732	1.012461	1.035916	1.00017	1.000016	1.000014
	GPSO	R_{avg}	1.0000407	1.0009582	1.008766	1.0282612	1.000364	1.000088	1.0000199
		R_{max}	1.0000518	1.0013935	1.0097839	1.0362575	1.000606	1.000235	1.0000567
800	SA	R_{avg}	1.00026	1.004415	1.016517	1.033841	1.000733	1.000183	1.000078
		R_{max}	1.000352	1.004862	1.021035	1.037611	1.001125	1.000362	1.000103
	GA	R_{avg}	1.000036	1.00667	1.038053	1.069994	1.012143	1.003916	1.001570
		R_{max}	1.000091	1.007716	1.045607	1.074328	1.017268	1.006536	1.002125
	I-L	R_{avg}	1.001087	1.001179	1.002329	1.014871	1.000172	1.000759	1.000235
		R_{max}	1.001886	1.001577	1.00434	1.022341	1.000858	1.002482	1.001174
	TS	R_{avg}	1.000000	1.001584	1.012993	1.031164	1.000014	1.000014	1.000002
		R_{max}	1.000000	1.001954	1.016	1.034914	1.000051	1.000069	1.000012
	GPSO	R_{avg}	1.0000273	1.0002623	1.0030067	1.0174544	1.000098	1.0000389	1.0000075
		R_{max}	1.000091	1.0003974	1.0039185	1.0209431	1.000188	1.0000705	1.0000129
900	SA	R_{avg}	1.000312	1.005794	1.018969	1.037928	1.001489	1.000321	1.000109
		R_{max}	1.000394	1.006735	1.020701	1.044494	1.001891	1.00042	1.000248
	GA	R_{avg}	1.000062	1.011499	1.052391	1.087316	1.023624	1.014302	1.004871
		R_{max}	1.000115	1.014396	1.065397	1.097871	1.029638	1.028271	1.005309
	I-L	R_{avg}	1.000772	1.000551	1.001911	1.008458	1.000401	1.000034	1.000000
		R_{max}	1.001178	1.000747	1.003417	1.018013	1.000619	1.000171	1.000000
	TS	R_{avg}	1.000000	1.001523	1.01131	1.027987	1.000024	1.000000	1.000002
		R_{max}	1.000000	1.001666	1.013694	1.034409	1.000045	1.000000	1.000011
	GPSO	R_{avg}	1.0000237	1.0002825	1.0032001	1.0164708	1.000224	1.0000243	1.0000281
		R_{max}	1.0000397	1.0003273	1.0047296	1.0219045	1.000456	1.0000971	1.0000969
1000	SA	R_{avg}	1.00033	1.007395	1.02317	1.050076	1.006872	1.000252	1.000277
		R_{max}	1.000437	1.008695	1.024848	1.06515	1.014001	1.000447	1.000445
	GA	R_{avg}	1.000122	1.01841	1.067022	1.108872	1.041892	1.021862	1.019433
		R_{max}	1.000146	1.020005	1.073896	1.133402	1.061253	1.028772	1.03123
	I-L	R_{avg}	1.000807	1.000566	1.001989	1.010157	1.000777	1.000414	1.000146
		R_{max}	1.001517	1.000894	1.003356	1.016946	1.002464	1.001269	1.000348
	TS	R_{avg}	1.000000	1.001561	1.011537	1.031613	1.000307	1.000031	1.00001
		R_{max}	1.000000	1.001893	1.013086	1.033813	1.000502	1.000066	1.00002
	GPSO	R_{avg}	1.0000145	1.0003875	1.0040583	1.0222069	1.00023	1.0000352	1.000018
		R_{max}	1.0000364	1.0004732	1.0045929	1.0252448	1.000449	1.0000882	1.0000397

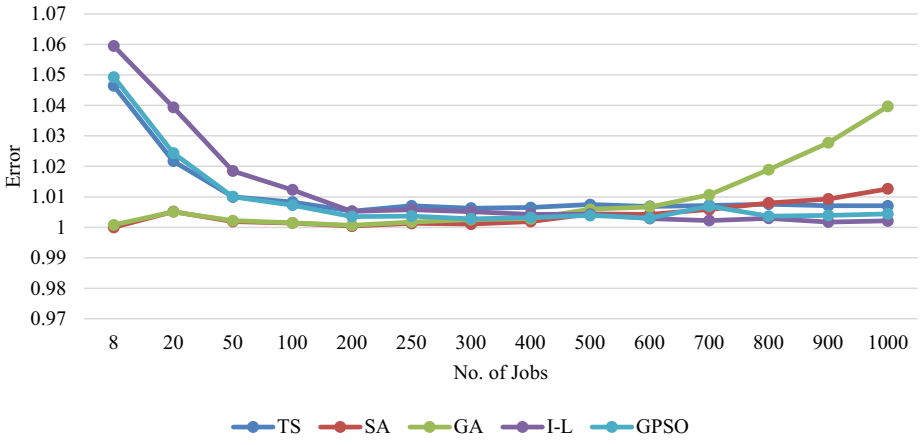

Fig. 2. Average values of the relation of error for all instances over all values of*L*.

- *n* denotes the number of jobs.
- R_{avg} denotes the average value of the error at the specified *L* values.
- R_{max} denotes the maximum value of the error among all instances at the specified *L* values.

Comparing the obtained results of the two designed algorithms to the three-benchmark algorithms from previous studies, the following summary can be given. As we can see from Table 3, it was hard to reach LB for all algorithms as the number of jobs increases except for some cases when *L* was equal to 0.1, 0.5, 1.8, 2. Instances with *L* values equal to 0.8, 1, or 1.5 were difficult to be solved to its minimum (reaching the LB) in all algorithms for all instances except for SA when the number of jobs was 8. It can be noted that the R_{avg} largest values of SA, GA, I-L, TS, and GPSO were 1.050076, 1.108872, 1.103816, 1.031613, and 1.035704, respectively. Among

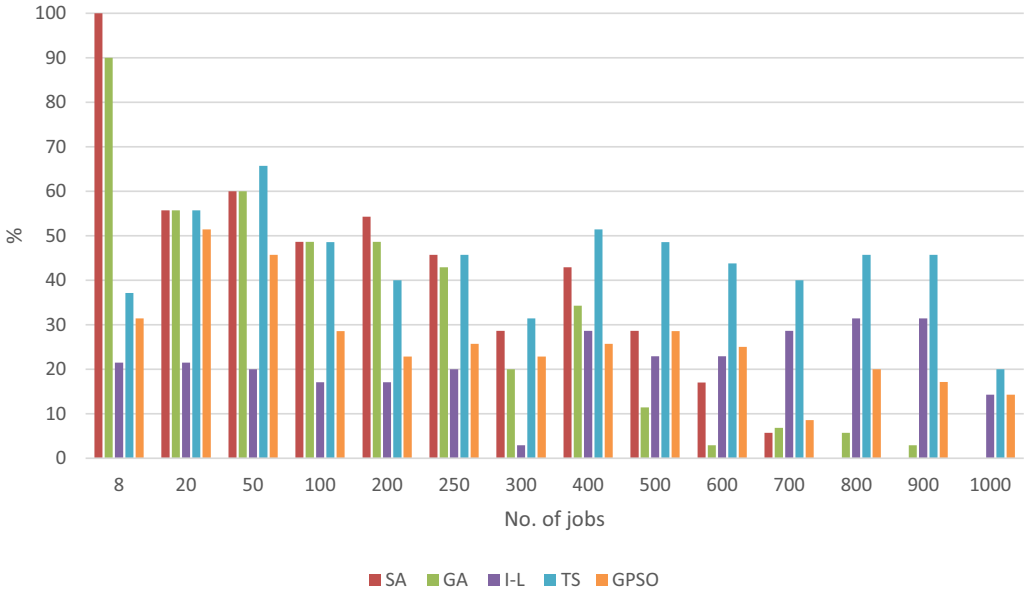

Fig. 3. Percentage of instances where the lower bound is obtained by particular algorithm.

the R_{avg} largest values, TS achieved the best one (1.031613) and GPSO came next while GA achieved the worst (1.108872). It is also noted that the R_{avg} largest values for I-L occurred when L is 0.8 and the number of jobs was 8 while the R_{avg} largest values for TS, SA, GA, and GPSO occurred when L is 1 and the number of jobs was 1000 (for TS, SA and GA) and 8 (for GPSO). This means that GPSO and I-L perform well for large instances.

From Fig. 2, the results of SA and GA were closer to each other for small instances ($n \leq 250$) except when $n = 8$ jobs where SA and GA performed better. GPSO, TS and I-L were closer to each other for small instances ($n \leq 100$), however TS and GPSO outperformed I-L. All metaheuristics outperformed I-L algorithm for $n < 200$. All algorithms performed the same when number of jobs n was between 200 and 600. It is worth noting that all algorithms are much closer for instances between 200 and 600 jobs. For $n \geq 700$ all metaheuristics outperformed GA. It worth notice that TS, GPSO, I-L, and SA performance were consistent among all instances greater than 200 jobs as we can see in Fig. 2.

Fig. 3 shows the percentage of instances (for all instances within each n number of jobs) where the lower bound is obtained by particular algorithm. Therefore, it gives an indication for the percentage of instances where the optimal value is acquired. The results of SA, GA, and I-L algorithms were obtained from Table 1, Hasani et al. (2016). In general, the percentage of reaching the LB was decreased with increasing in number of jobs. It can be seen that SA reached LB for all L values at $n = 8$ as the percentage is 100%. However, this percentage becomes smaller as the number of instances increased. It becomes zero for all instances when the number of jobs was greater than 700. TS outperformed all algorithms for all instances greater than 200 jobs and GPSO reached better percentages than SA and GA when number of jobs was more than 400. Note that a comparison with SA, GA, and I-L in term of CPU time was not conducted, because the CPU times was set to be the same in all experimentations ($(\frac{300}{8}) * n$ for small-scale size of the problems and 3600 for the large-scale).

5. Conclusions and future work

This paper considers the scheduling problem of identical parallel machines with a single server to minimize the makespan. Two metaheuristics have been proposed, which are the TS and GPSO. The performance of the two proposed algorithms was assessed using the same instances generated in Hasani et al. (2016) and compared to the three algorithms proposed by Hasani et al. (2016). The three-benchmark algorithms were SA, GA and I-L. The comparison study was conducted on a large set of instances with up to 1000 jobs. The computational results have shown that the two proposed algorithms (TS and GPSO) performed well in term of a used error, which represents the deviation from the known lower bound especially for medium- and large-scale instances compared to SA and GA. Moreover, TS outperformed all algorithms in term of reaching the lower bound for all instances except when the number of jobs are 8 and 200. The sequencing strategy of the jobs in such a way that the idle times of machine and waiting times of server are decreased converted to be primary for a higher quality of the developed algorithms.

In future research, it is meant to introduce many types of heuristics and metaheuristics for the makespan minimization problem with an arbitrary number of machines and a single server. Moreover, an interesting way for future work is to extend the current research to more advanced objective functions such as total tardiness and total completion time.

6. Data availability

The data used to support the findings of this study are included within the article.

Conflict of interest

The authors declare that there is no conflict of interest regarding the publication of this paper.

Funding statement

This work was supported by the research center of the college of engineering in King Saud University.

Acknowledgments

The authors would like to thank Engineering Optimization Journal for giving permission to use Table 1 and Fig. 9 with license numbers (446639146538) and (4466391338522), respectively, from Hasani, K., Kravchenko, S. A., & Werner, F. (2016). Minimizing the makespan for the two-machine scheduling problem with a single server: Two algorithms for very large instances. Engineering Optimization, 48(1), 173–183. https://doi.org/10.1080/0305215X.2015.1005083.

References

Abdekhodaee, A.H., Wirth, A., 2002. Scheduling parallel machines with a single server: some solvable cases and heuristics. Comput. Oper. Res. 29 (3), 295–315. https://doi.org/10.1016/S0305-0548(00)00074-5.

Abdekhodaee, A.H., Wirth, A., Gan, H.-S., 2006. Scheduling two parallel machines with a single server: the general case. Comput. Oper. Res. 33 (4), 994–1009. https://doi.org/10.1016/J.COR.2004.08.013.

Allahverdi, A., Ng, C.T., Cheng, T.C.E., Kovalyov, M.Y., 2008. A survey of scheduling problems with setup times or costs. Eur. J. Oper. Res. 187 (3), 985–1032. https://doi.org/10.1016/j.ejor.2006.06.060.

Arnaout, J.-P., 2017. Heuristics for the two-machine scheduling problem with a single server. Int. Trans. Oper. Res. 24 (6), 1347–1355. https://doi.org/10.1111/itor.12302.

Della Croce, F., Narayan, V., Tadei, R., 1996. The two-machine total completion time flow shop problem. Eur. J. Oper. Res. 90 (2), 227–237. https://doi.org/10.1016/0377-2217(95)00351-7.

Gan, H.-S., Wirth, A., Abdekhodaee, A., 2012. A branch-and-price algorithm for the general case of scheduling parallel machines with a single server. Comput. Oper. Res. 39 (9), 2242–2247. https://doi.org/10.1016/J.COR.2011.11.007.

Glass, C.A., Shafransky, Y.M., Strusevich, V.A., 2000. Scheduling for parallel dedicated machines with a single server. Nav. Res. Logist. 47 (4), 304–328. https://doi.org/10.1002/(SICI)1520-6750(200006)47:4<304::AID-NAV3>3.0.CO;2-1.

Glover, F., 1989. Tabu search—Part I. ORSA J. Comput. 1 (3), 190–206. https://doi.org/10.1287/ijoc.1.3.190.

Glover, F., Laguna, M., 2013. Tabu search. In: Handbook of Combinatorial Optimization. Springer New York, New York, NY, pp. 3261–3362. https://doi.org/10.1007/978-1-4419-7997-1_17.

Hall, N.G., Potts, C.N., Sriskandarajah, C., 2000. Parallel machine scheduling with a common server. Discrete Appl. Math. 102 (3), 223–243. https://doi.org/10.1016/S0166-218X(99)00206-1.

Hasani, K., Kravchenko, S.A., Werner, F., 2014b. Simulated annealing and genetic algorithms for the two-machine scheduling problem with a single server. Int. J. Prod. Res. 52 (13), 3778–3792. https://doi.org/10.1080/00207543.2013.874607.

Hasani, K., Kravchenko, S.A., Werner, F., 2014a. Block models for scheduling jobs on two parallel machines with a single server. Comput. Oper. Res. 41, 94–97. https://doi.org/10.1016/J.COR.2013.08.015.

Hasani, K., Kravchenko, S.A., Werner, F., 2016. Minimizing the makespan for the two-machine scheduling problem with a single server: two algorithms for very large instances. Eng. Optim. 48 (1), 173–183. https://doi.org/10.1080/0305215X.2015.1005083.

Jia, Y., Qu, J., Wang, L., 2016. In: A Novel Particle Swarm Optimization Algorithm for Permutation Flow-Shop Scheduling Problem. Springer, Cham, pp. 676–682.

Kennedy, J., Eberhart, R., 1995. Particle swarm optimization. In: Neural Networks, 1995. Proceedings. IEEE International Conference On, pp. 1942–1948. https://doi.org/10.1109/ICNN.1995.488968.

Kim, M.Y., Lee, Y.H., 2012. MIP models and hybrid algorithm for minimizing the makespan of parallel machines scheduling problem with a single server. Comput. Oper. Res. 39 (11), 2457–2468. https://doi.org/10.1016/j.cor.2011.12.011.

Kravchenko, S.A., Werner, F., 1997. Parallel machine scheduling problems with a single server. Math. Comput. Modell. 26 (12), 1–11. https://doi.org/10.1016/S0895-7177(97)00236-7.

Moraglio, A., Di Chio, C., Togelius, J., Poli, R., 2008. Geometric particle swarm optimization. J. Artif. Evol. Appl. 2008, 1–14. https://doi.org/10.1155/2008/143624.

Reinelt, G. (Gerhard), Gerhard, 1994. The Traveling Salesman: Computational Solutions for TSP Applications. Springer-Verlag.

Siddhartha, Sharma, N., Varun, 2012. A particle swarm optimization algorithm for optimization of thermal performance of a smooth flat plate solar air heater. Energy 38 (1), 406–413. https://doi.org/10.1016/J.ENERGY.2011.11.026.

Talbi, E., 2009. From design to implementation. Main. John Wiley & Sons https://books.google.com.sa/books?hl=en&lr=&id=SIsa6zi5XV8C&oi=fnd&pg=PR7&dq=Talbi,+E.-G.+2009.+Metaheuristics:+from+design+to+implementation,+John+Wiley+%26+Sons.&ots=-aRLpTfrBx&sig=WiW7O-xfmYRUps_SsSGypehrX3I&redir_esc=y#v=onepage&q=Talbi%2C E.-G.

Wang, K., & Huang, L. (2003). Particle swarm optimization for traveling salesman problem. Machine Learning and . . ., (November), 1583–1585. https://doi.org/10.1109/ICMLC.2003.1259748.

Zhao, Y.W., Wu, B., Wang, W.L., Ma, Y.L., Wang, W.A., Sun, H., 2004. Particle swarm optimization for vehicle routing problem with time windows. Mater. Sci. Forum 471–472, 801–805. https://doi.org/10.4028/www.scientific.net/MSF.471-472.801.